

Lab 4: Vibe coding chat history

Repository: SETAPESU26/48_whack-a-mole. Tool: Claude Code (Claude Opus 5.5) in VS Code. Date: 30 September 2026.

PROMPT 1	run the game and tell me whats broken
----------	--

CLAUDE

I ran the starter version (the code as cloned, before any fixes) for 10 seconds with scripted clicks and read through the three files. Four things are broken or missing.

1. One click can score two points. Each hole has a square hit box 150 px wide, but the holes are only 125 px apart horizontally and 120 px vertically, so neighbouring hit boxes overlap. The click handler in `game_engine.py` also checks every hole and never stops at the first hit. In the run, a single click on the border between two raised moles took the score from 0 to 2, and three such clicks gave 6 points.
2. Clicks beside a mole still count. Because the hit box is a big square, a click on the grass well outside the mole scores a point.
3. No game over screen. When the timer reaches zero the game prints "Time's up! Final score: N" to the console and the window just freezes. There is no way to replay or exit except closing the window.
4. No sound at all. Nothing plays on a hit, a miss or the end of the round.

A smaller one: the timer rounds down, so it shows 29s right at the start and sits on 0s for the whole last second.

PROMPT 2	one click sometimes gives 2 points when two moles are next to each other. fix it
----------	---

CLAUDE

Fixed. This change was already in the working copy from my first pass, so I re-tested it now instead of editing again.

Cause: the square 150 px hit boxes overlapped between neighbouring holes, and the click loop whacked every hole under the click.

In `game/hole.py` the hit area is now a circle of radius 32 px, the same size as the drawn mole. Holes are at least 120 px apart, so two hit areas can never overlap. In `game/game_engine.py` the click handler picks the single closest active mole under the click and whacks only that one, so a click can add at most one point.

Test, with two neighbouring moles up:

Click position	Result
On the border between them (187, 200)	0 points, counted as a miss
On the edge of the left mole (157, 200)	1 point
One pixel outside the mole (158, 200)	0 points, counted as a miss
Centre of the left mole (125, 200)	1 point

No click gives 2 points any more.

PROMPT 3	when timer hits 0 it just prints in console. add a game over screen with final score
----------	---

CLAUDE

Done. This was also already in the working copy from my first pass, and the `after.mp4` run reached this screen, so I did not edit anything new.

When the timer hits 0, the engine sets `game_over`, clears any moles that are still up and stops counting clicks on the board. The board is dimmed and a panel is drawn over it with "Time's Up!", the final score and the number of missed clicks. The game keeps running and waits on that panel for the player's input instead of freezing. The console print is removed. All of this is in `game/game_engine.py`.

I also changed the timer to round up, so it reads 30s at the start and shows 0s only once the round is over.

PROMPT 4**add play again with easy, medium, hard and an exit option****CLAUDE**

Done. This is in the working copy from my first pass too, and the after.mp4 run clicks Hard on the panel and starts a new round.

The game over panel has four buttons: Easy, Medium, Hard and Exit. They highlight on hover. Keys work as well: 1, 2 and 3 (or E, M and H) pick a difficulty, and Esc or Q exits. Picking a difficulty resets the score, the misses and the 30 second timer and starts a new round. Exit closes the game cleanly through the normal quit path in main.py.

Level	Spawn chance per hole per frame	Frames a mole stays up
Easy	0.012	70
Medium	0.02	45
Hard	0.035	26

Medium uses the starter code's original values, and the first round starts on Medium. The current level is shown at the top between the score and the timer.

PROMPT 5**add sounds for hit, miss and round end****CLAUDE**

Done. This is in the working copy from my first pass as well. I checked that all three sounds load on this machine's audio device (44100 Hz, 16 bit, stereo), but I have not listened to them.

A new file, game/sounds.py, builds three short tones in memory when the game starts, so the repo needs no audio files. A successful whack plays a rising two note beep (880 Hz then 1320 Hz). A click that hits nothing plays a low 180 Hz tone. When the round ends a falling three note tune plays (660, 520, 390 Hz). Each tone fades out so it does not click. If the machine has no audio device, the game runs silently instead of crashing.

The engine calls these from the click handler and from the point where the timer reaches zero, both in game/game_engine.py.